Améliorer la qualité du code

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Les exceptions **3**

A. Les exceptions 3

B. Exercice : Quiz 5

III. Bonnes Pratiques en Python **6**

A. Bonnes Pratiques en Python 6

B. Exercice : Quiz 8

IV. Utilitaires de Formatage, Analyse et Gestion du Code **11**

A. Utilitaires de Formatage, Analyse et Gestion du Code 11

B. Exercice : Quiz 12

V. Essentiel **14**

VI. Auto-évaluation **14**

A. Exercice 14

B. Test 15

Solutions des exercices **16**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Contexte

Bienvenue dans ce cours sur l'amélioration du code en Python. L'objectif est de vous fournir des techniques et des bonnes pratiques pour écrire un code de qualité, maintenable et efficace.

Imaginez un conducteur évitant un obstacle de manière contrôlée. De même, la gestion des exceptions en Python permet de gérer les erreurs proprement, en évitant les plantages et en assurant la continuité du programme. Afin de pouvoir toujours maintenir un niveau de qualité élevé et éviter les plantages, il est nécessaire de rédiger un document professionnel bien formaté. En Python, suivre PEP 8 (guide des bonnes pratiques) assure un code lisible et cohérent, facilitant la collaboration et la compréhension du code.

Enfin, pour s'assurer que tout est fait correctement, nous utiliserons des outils comme black et flake8 qui nous aideront à maintenir un code propre et exempt d'erreurs, garantissant des standards de qualité élevés.

II. Les exceptions

A. Les exceptions

Définition

Les exceptions en Python sont des événements qui interrompent le traitement d'un programme en raison d'une condition inhabituelle ou inattendue. Il est nécessaire d'apprendre à les gérer plutôt que de laisser le programme planter brutalement. En Python, il est possible de séparer le code des erreurs, augmentant ainsi la lisibilité.

Nous avons tous déjà rencontré un programme dans lequel une erreur s'est produite, à tel point qu'il nous a « *éjectés* » de notre progression. Cette expérience extrêmement frustrante peut empêcher un utilisateur de revenir sur votre application et ternir votre réputation. Il est donc fondamental d'avoir un programme qui gère les erreurs de manière performante et pertinente.

Cette gestion ne doit pas s'appliquer à tout et n'importe quoi. Il y a 4 grandes thématiques : si vous rencontrez l'une de ces situations, il est vivement recommandé d'effectuer une gestion d'erreur :

- **Saisie utilisateur :** ne jamais faire confiance à un utilisateur
- **Lecture de fichier, appel à une ressource externe :** la ressource ou le fichier, peut, ne pas exister
- **Calcul :** il est nécessaire de vérifier que vos calculs se déroulent correctement. Par exemple, la division par zéro nécessite une gestion particulière.
- **Règle métier :** certaines entreprises imposent des règles métier dans leurs applications. Ainsi, si une règle métier n'est pas respectée, l'entreprise peut déclencher une exception pour empêcher l'utilisateur de continuer son action.

La gestion des exceptions en Python est effectuée à l'aide des blocs `try`, `except`, `else`, et `finally`.

```python
try:
# Code pouvant générer une exception
i: float = 1/0
except ZeroDivisionError as e:
# Code exécuté en cas d'exception
print("division par 0, c'est impossible")
else:
# Code exécuté si aucune exception n'a été levée
print("la division s'est bien passée")
finally:
# Code exécuté quoi qu'il arrive (qu'il y ait eu une exception ou non)
print("traitement terminé")
```

- `try` : le bloc où le code potentiellement problématique est placé.
- `except` : le bloc où le code de gestion des erreurs est placé.
- `else` : le bloc où le code est exécuté si aucune exception n'a été levée.
- `finally` : le bloc où le code est toujours exécuté, qu'il y ait eu une exception ou non.

En Python, les exceptions sont gérées en utilisant des blocs try et except. Lorsque le code dans le bloc try génère une exception, le contrôle est immédiatement transféré au bloc except correspondant. Si aucune exception n'est levée, le code dans le bloc except est ignoré. De plus, les blocs else et finally peuvent être utilisés pour ajouter du code qui doit être exécuté respectivement si aucune exception n'est levée ou dans tous les cas. Le bloc finally sera exécuté que le bloc except soit activé ou non, contrairement à else, qui ne s'exécutera que si le bloc except n'est pas activé.

Méthode

Voyons maintenant une autre gestion d'erreur. Nous allons gérer deux possibilités :

- Si la valeur n'est pas correcte (saisir une string par exemple à la place d'un int).
- Si la division par 0 se produit, comment la gérer.

```
1 try:
2 num_un = float(input("Entrez le premier nombre : "))
3 num_deux = float(input("Entrez le deuxième nombre: "))
4 resultat = num_un / num_deux
5 print("Le résultat de la division est : {resultat}")
6 except ValueError:
7 print("Erreur : Veuillez entrer un nombre valide.")
8 except ZeroDivisionError:
9 print("Erreur : Division par zéro non autorisée.")
10 except Exception :
11   print(" Non géré ")
12 finally:
13 print("Fin de l'opération.")
```

Python va entrer dans le premier bloc except qu'il rencontre (de haut en bas), puis passer dans le bloc finally. Il n'exécutera qu'un seul bloc except, ce qui signifie que s'il entre dans ValueError, il n'ira pas dans Exception (qui est l'erreur générique en Python). C'est pourquoi il est recommandé de placer les blocs except du plus spécifique au plus général.

Jusqu'à présent, nous avons utilisé des exceptions fournies par Python, mais nous pouvons également définir nos propres exceptions en créant de nouvelles classes d'exception. Pour cela, il suffit de créer une classe qui hérite de la classe Exception ou de l'une de ses sous-classes.

Dans un premier temps, nous allons concevoir notre classe contenant notre exception.

```
1 class MonExceptionPersonnalisee(Exception) :
2 def __init__(self,message) :
3   super().__init__(message)
4   self.message = message
```

Ensuite, nous pouvons l'utiliser. Pour la lever (la déclencher), nous devons utiliser le mot-clé raise. Dans l'exemple proposé, si b est égal à 0, nous allons lever une exception que nous avons développée afin de prévenir la division par zéro, qui est interdite.

Tous droits réservés à STUDI - Reproduction interdite

```
1 def diviser(a, b):
2 if b == 0:
3 raise MonExceptionPersonnalisee("La division par zéro n'est pas autorisée.")
4 return a / b
5
6 try:
7 resultat = diviser (10, 0)
8 print(resultat)
9 except MonExceptionPersonnalisee as e:
10 print("Erreur : {e.message}")
11 finally:
12 print("Opération de division terminée.")
```

Dans cet exemple, la fonction `diviser` lève une `MonExceptionPersonnalisee` si une tentative de division par zéro est effectuée. Le bloc `try` tente d'exécuter la fonction, et le bloc `except` capture l'exception personnalisée pour afficher un message d'erreur approprié. Le bloc `finally` s'exécute toujours, indiquant la fin de l'opération.

B. Exercice : Quiz

[solution n°1 p.17]

Question 1

Qu'est-ce qu'une exception ?

- Une condition inhabituelle qui interrompt le flux normal du programme
- Une variable non initialisée
- Une boucle infinie
- Une fonction sans paramètre

Question 2

Quel bloc est toujours exécuté, qu'il y ait une exception ou non ?

- try
- except
- else
- finally

Question 3

Que fait le bloc `else` dans une gestion d'exception ?

- Il contient du code qui est exécuté si une exception est levée.
- Il contient du code qui est exécuté si aucune exception n'est levée.
- Il contient du code qui est exécuté après le bloc `finally`.
- Il contient du code qui est exécuté avant le bloc `try`.

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Quel type d'exception est levée lorsque l'on tente de convertir une chaîne non numérique en entier ?

- IOError
- ValueError
- TypeError
- KeyError

Question 5

Pourquoi utiliser des exceptions ?

- Pour arrêter le programme dès qu'une erreur survient.
- Pour gérer les erreurs de manière contrôlée et maintenir la lisibilité du code.
- Pour ignorer les erreurs.
- Pour augmenter la complexité du code.

III. Bonnes Pratiques en Python

A. Bonnes Pratiques en Python

En Python, la PEP est le guide des bonnes pratiques. Il est important de respecter les principes qu'elle expose afin de produire un code de qualité et d'assurer une bonne lisibilité ainsi que de la cohérence. Cela permet de faciliter la collaboration et la maintenance du code. La PEP 8, disponible ici[1], regorge d'innombrables bonnes pratiques. Voyons ensemble les plus classiques :

- **Indentation** : utilisez 4 espaces par niveau d'indentation.
- **Longueur des lignes** : limitez les lignes à 79 caractères.
- **Espaces autour des opérateurs** : utilisez des espaces autour des opérateurs et après les virgules.
- N**oms des variables et fonctions** : utilisez le snakecase pour les variables et les fonctions (ex. : `ma_variable`, `ma_fonction`).
- **Noms des classes** : utilisez le style CamelCase pour les noms de classes (ex. : `MaClasse`).
- **Commentaires** : utilisez des commentaires pour expliquer pourquoi un code fait quelque chose, pas seulement ce qu'il fait.

Exemple

Le code ci-dessous fonctionne et est séduisant en apparence. Mais, il comporte beaucoup d'imperfections :

```
1 def calculersomme(a, b): return a + b
2
3 class comptebancaire:
4 def __init__(self, solde_initial):
5 self.solde = solde_initial
6 def deposer(self, montant): self.solde + montant
7 def retirer(self, montant):
8 if montant <= self.solde:
9   self.solde -= montant
10 else:
11 print("Solde insuffisant.")
```

1. https://peps.python.org/pep-0008/

Tous droits réservés à STUDI - Reproduction interdite

Analysons en détail ce code avec les recommandations de la PEP :

1. **Nom de la fonction et de la classe**
   - **Mauvaise pratique** : `calculersomme` (nom en camelCase) et `comptebancaire` (nom en minuscule).
   - **Pourquoi c'est mauvais** : Python utilise le style snake_case pour les noms de fonctions et CamelCase pour les noms de classes. Respecter ces conventions rend le code plus lisible et cohérent avec les autres codes Python.
   - **Bonne pratique** : 'calculer_somme' pour la fonction, et, 'CompteBancaire' pour la class
2. **Espacement et lisibilité**
   - **Mauvaise pratique** : `def calculersomme(a,b): return a+b`
     - Pourquoi c'est mauvais :
     - Manque d'espaces après les virgules, rendant le code plus difficile à lire.
     - Le retour est sur la même ligne que la définition de la fonction, ce qui nuit à la lisibilité.
   - **Bonne pratique** :

     `def calculer_somme(a, b):

     return a + b`
   - **Pourquoi c'est bien :**
     - Les espaces après les virgules et autour de l'opérateur `+` améliorent la lisibilité.
     - La séparation de la définition de la fonction et du retour sur des lignes distinctes rend le code plus clair.
3. **Indentation et structure**
   - **Mauvaise pratique** : `class comptebancaire:`
     - Pourquoi c'est mauvais :
     - Mauvaise indentation des méthodes, rendant le code difficile à lire et à maintenir.
     - Les définitions de méthodes et les déclarations sont sur une seule ligne, ce qui nuit à la clarté.
   - **Bonne pratique** : indentation de chaque méthode dans une class afin qu'elles soient incluses et lisible

En appliquant tous les éléments, voici le code final qui est une bonne pratique :

```
1 def calculer_somme(a, b):
2 return a + b
3 class CompteBancaire:
4 def __init__(self, solde initial):
5 self.solde = solde_initial
6 def deposer(self, montant):
7 self.solde += montant
8 def retirer(self, montant):
9 if montant <= self.solde:
10   self.solde -= montant
11 else:
12 print("Solde insuffisant.")
```

Fondamental

Nous pouvons aller plus loin dans notre démarche de qualité : la PEP recommande l'utilisation des commentaires. Les commentaires ainsi que la documentation sont essentiels pour la maintenabilité du code. Ils doivent être clairs et concis, expliquant le pourquoi du code plutôt que le comment. Les docstrings permettent de documenter les modules, classes et fonctions.

Tous droits réservés à STUDI - Reproduction interdite

Un premier exemple serait de documenter une fonction.

```python
def addition(a, b):
"""
Calcule la somme de deux nombres.
:param a: premier nombre
:param b: deuxième nombre
:return: somme de a et b
"""
return a + b
```

On peut observer, trois éléments pour une bonne documentation en Python :

- La docstring commence par une brève description de la fonction
- Les paramètres ‘a’ et ‘b’ sont décrits avec le type attendu et leur rôle.
- Le type de retour et la signification de la valeur retournée sont également documentés.

Si l'on souhaite adapter une docstring à une classe Python, cela est possible mais nécessite plus d’informations. Par exemple, il faut décrire les attributs ainsi que les méthodes que possède la classe, suivis de l’utilité de celle-ci.

```python
class CompteBancaire:
"""

Une classe pour représenter un compte bancaire.
Attributes:
solde (float): Le solde initial du compte.
Methods:
deposer (montant):
Dépose un montant sur le compte.
retirer (montant):
Retire un montant du compte s'il y a suffisamment de solde.
"""
```

B. Exercice : Quiz

[solution n°2 p.18]

Question 1

Quelle est la recommandation de la PEP 8 pour l'indentation en Python ?

- ○ Utiliser 2 espaces par niveau d'indentation
- ○ Utiliser 4 espaces par niveau d'indentation
- ○ Utiliser des tabulations pour l'indentation
- ○ Utiliser 8 espaces par niveau d'indentation

Question 2

Quelle est la longueur maximale recommandée pour une ligne de code en Python selon la PEP 8 ?

- ○ 60 caractères
- ○ 72 caractères
- ○ 79 caractères
- ○ 100 caractères

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Quel style de nommage est recommandé pour les fonctions en Python ?

- camelCase
- PascalCase
- snake_case
- kebab-case

Question 4

Quel est le style de nommage recommandé pour les classes en Python ?

- camelCase
- PascalCase (CamelCase)
- snake_case
- UPPER_CASE

Question 5

Pourquoi est-il important d'utiliser des espaces autour des opérateurs et après les virgules ?

- Pour améliorer la performance du code
- Pour respecter les conventions de syntaxe de Python
- Pour améliorer la lisibilité du code
- Pour réduire la longueur du code

Question 6

Quel est le problème avec le code suivant : `def calculersomme(a,b): return a+b` ?

- Il manque des commentaires
- Il utilise des tabulations au lieu d'espaces
- Il manque des espaces autour des opérateurs et des virgules
- Il utilise un nom de fonction incorrect

Question 7

Dans l'exemple suivant, quel est le problème avec la définition de la classe `comptebancaire` ?

```
1 class comptebancaire:
2     def __init__(self, solde_initial):
3         self.solde = solde_initial
```

- Le nom de la classe doit être en CamelCase
- Les méthodes ne sont pas correctement indentées
- Le nom de la classe doit être en snake_case
- Il manque un docstring pour la classe

Question 8

Tous droits réservés à STUDI - Reproduction interdite

Quel est le principal objectif des docstrings dans le code Python ?

- Décrire chaque ligne de code
- Expliquer la logique du code
- Documenter les modules, classes et fonctions
- Optimiser les performances du code

Question 9

Pourquoi est-il recommandé de documenter les paramètres et le type de retour dans une docstring ?

- Pour améliorer la performance du code
- Pour faciliter la compréhension et l'utilisation de la fonction par d'autres développeurs
- Pour répondre aux exigences syntaxiques de Python
- Pour réduire la taille de la fonction

Question 10

Dans l'exemple suivant, quel est l'objectif principal des sections "Attributes" et "Methods" dans la docstring d'une classe ?

```python
class CompteBancaire:
    """
    Une classe pour représenter un compte bancaire.

    Attributes:
        solde (float): Le solde initial du compte.

    Methods:
        deposer(montant):
            Dépose un montant sur le compte.
        retirer(montant):
            Retire un montant du compte s'il y a suffisamment de solde.
    """
```

- Décrire la logique interne de chaque méthode
- Fournir une documentation claire des attributs et des méthodes de la classe
- Optimiser les performances de la classe
- Assurer que le code est conforme à la PEP 8

Tous droits réservés à STUDI - Reproduction interdite

IV. Utilitaires de Formatage, Analyse et Gestion du Code

A. Utilitaires de Formatage, Analyse et Gestion du Code

Dans cette section, nous allons explorer divers utilitaires et outils qui peuvent grandement améliorer la qualité, la lisibilité et la maintenabilité de votre code Python. Utiliser ces outils permet de garantir que le code respecte les normes et conventions, est exempt d'erreurs courantes et bien documenté. Il existe des outils pour le formatage automatique du code :

- L'analyse statique
- La vérification de sécurité
- La gestion des dépendances
- Les tests

Définition

Le formatage en Python est extrêmement important puisqu'il permet de structurer notre approche. À ce titre, black est un formateur de code que nous pouvons installer via pip install black, et nous pouvons même l'ajouter dans notre configuration PyCharm afin d'automatiser son action. Le formatage permet de mettre le bon nombre de lignes et d'espaces dans votre code, ce qui est fondamental pour être conforme à la PEP.

Méthode

Installation dans PyCharm :

1. Ouvrez PyCharm.
2. Allez dans `File > Settings > Tools > External Tools`.
3. Cliquez sur `+` pour ajouter un nouvel outil.
4. Configurez comme suit:
    - Name: Black
    - Program: `black`
    - Arguments: `$FilePathRelativeToProjectRoot$`
    - Working directory: `$ProjectFileDir$`

Avec PyCharm, l'exécution de black est automatique lors de l'enregistrement d'un fichier. En revanche, manuellement, il faut démarrer un shell si vous êtes sous Linux ou une invite de commande sous Windows.

```
Black nom_fichier.py
```

Deux autres outils peuvent nous aider : isort et flake8. Le premier permet de trier et de retirer les importations inutiles dans nos fichiers, tandis que le second permet de détecter les erreurs de style et de syntaxe. Flake8 est intéressant, mais il est souvent remplacé par pylint, qui est plus complet et met en évidence les mauvaises pratiques dans le code.

Les trois outils s'installent via pip :

pip install isort flake8 pylint

Pour l'utilisation, ils s'exécutent tout simplement en passant par un shell, identique à black.

Python dispose également d'un outil permettant d'effectuer des analyses de sécurité, nommé bandit, installable également via pip. Bandit fonctionne de la même manière que black.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Voici un exemple de sortie que nous pouvons obtenir :

```
[bandit] INFO  running on Python 3.8.10
Run started:2023-04-15 10:29:15.962378

Test results:
>> Issue: [B101:assert_used] Use of assert detected. The enclosed code will be removed when compiling to optimised byte code.
   Severity: Low   Confidence: High
   Location: nom_du_fichier.py:12
   More Info: https://bandit.readthedocs.io/en/latest/plugins/b101_assert_used.html
11
12         assert user.is_admin
13         self.user = user
```

Figure 1 : résultat d'exécution de bandit

B. Exercice : Quiz

[solution n°3 p.21]

Question 1

Quel est l'objectif principal des outils de formatage automatique en Python comme 'black' ?

- Améliorer les performances du code
- Assurer la conformité du code avec la PEP 8
- Identifier les bugs dans le code
- Ajouter des fonctionnalités au code

Question 2

Quel est le rôle de 'isort' parmi les outils mentionnés dans la section ?

- Corriger les erreurs de syntaxe
- Analyser la sécurité du code
- Trier et retirer les importations inutiles
- Formater le code

Question 3

Quel outil est recommandé pour vérifier les erreurs de style et de syntaxe en Python ?

- black
- bandit
- isort
- flake8

Question 4

Tous droits réservés à STUDI - Reproduction interdite

Quel outil est souvent préféré à 'flake8' pour une analyse plus complète des pratiques de codage ?

- pylint
- black
- isort
- bandit

Question 5

Comment installe-t-on 'black' pour formater le code Python ?

- Via un package manager spécifique à PyCharm
- Via pip avec la commande `pip install python-black`
- Via un téléchargement manuel
- Via un script Python

Question 6

Quel outil mentionné est utilisé pour effectuer des analyses de sécurité sur du code Python ?

- flake8
- isort
- bandit
- pylint

Question 7

Quel est le comportement par défaut de 'black' lorsqu'il est configuré dans PyCharm ?

- Il s'exécute automatiquement lors de l'enregistrement du fichier
- Il doit être lancé manuellement via le terminal
- Il formate uniquement les commentaires
- Il ne s'exécute que sur demande

Question 8

Quel est l'avantage principal d'utiliser des outils comme 'flake8' et 'pylint' ?

- Ils ajoutent des fonctionnalités au code
- Ils réduisent la taille du fichier de code
- Ils améliorent la lisibilité et la qualité du code en identifiant les erreurs et les mauvaises pratiques
- Ils accélèrent l'exécution du code

Question 9

Tous droits réservés à STUDI - Reproduction interdite

9. Quel est le format de la commande pour exécuter 'black' manuellement sur un fichier Python ?

- `black nom_fichier.py`
- `python black nom_fichier.py`
- `black --run nom_fichier.py`
- `python -m black nom_fichier.py`

Question 10

Comment installe-t-on les outils 'isort', 'flake8', et 'pylint' ?

- Via un package manager spécifique à chaque outil
- Via pip avec la commande `pip install isort flake8 pylint`
- Via un téléchargement manuel
- Via une commande intégrée à Python

V. Essentiel

La gestion des exceptions est cruciale pour écrire un code robuste et résilient. En Python, les exceptions permettent de gérer les erreurs de manière contrôlée, évitant ainsi les plantages imprévus et facilitant le débogage. Pour tirer pleinement parti des exceptions, il est recommandé d'utiliser des exceptions spécifiques plutôt que générales, de fournir des messages d'erreur clairs et informatifs, et d'utiliser les blocs finally pour les opérations de nettoyage qui doivent être exécutées quelles que soient les circonstances.

Le respect des conventions PEP 8 est également fondamental pour assurer un code lisible, cohérent et facile à maintenir. PEP 8 est le guide de style officiel pour Python et définit des conventions qui améliorent la lisibilité du code. Il est essentiel d'adopter des conventions de nommage appropriées, telles que le snake_case pour les fonctions et le CamelCase pour les classes. De plus, maintenir une bonne indentation et un espacement corrects, ainsi que documenter le code avec des docstrings, contribue grandement à la clarté et à la maintenabilité du code.

Afin de vous aider à respecter la PEP 8, il existe des outils de formatage de code. L'outil black formate automatiquement le code pour respecter les conventions de style, garantissant ainsi une cohérence et une lisibilité améliorées. isort trie les importations de manière cohérente, facilitant la gestion des dépendances. pylint analyse le code pour identifier les erreurs et les mauvaises pratiques, offrant des suggestions pour améliorer la qualité du code.

VI. Auto-évaluation

A. Exercice

Question 1

[solution n°4 p.23]

Vous devez écrire une fonction Python division_securisee (a, b) qui prend deux arguments a et b et qui renvoie le résultat de la division a / b. Si b est égal à zéro, la fonction doit lever une exception personnalisée DivisionParZeroException avec le message « *La division par zéro n'est pas autorisée* ». Écrivez cette fonction, ainsi que la classe de l'exception personnalisée.

Question 2

[solution n°5 p.24]

Ecrivez un code qui utilise un bloc try, except, else, et finally pour appeler la fonction division_securisee(10, 0), capturer l'exception personnalisée, afficher un message d'erreur, et indiquer que l'opération de division est terminée.

Tous droits réservés à STUDI - Reproduction interdite

B. Test

Exercice 1 : Quiz

[solution n°6 p.24]

Question 1

Que signifie "ne jamais faire confiance à l'utilisateur" dans le contexte de la gestion d'erreurs ?

- L'utilisateur doit être exclu du test du programme
- Les entrées de l'utilisateur doivent toujours être vérifiées et validées pour éviter les erreurs
- Le programme doit bloquer toute saisie utilisateur
- L'utilisateur doit être formé pour éviter les erreurs

Question 2

Comment créer une exception personnalisée en Python ?

- En utilisant la classe `BaseException`
- En héritant de la classe `Exception`
- En utilisant le mot-clé `custom`
- En utilisant `try-except` sans `raise`

Question 3

Pourquoi utiliser un bloc `else` dans une gestion d'exceptions ?

- Pour gérer les erreurs inconnues
- Pour exécuter du code seulement si aucune exception n'a été levée
- Pour nettoyer les ressources après une exception
- Pour remplacer le bloc `except`

Question 4

Dans quel cas utiliseriez-vous un bloc `finally` en Python ?

- Pour ajouter une exception au programme
- Pour exécuter du code uniquement si une exception est levée
- Pour exécuter du code quel que soit le résultat des blocs `try` et `except`
- Pour ignorer une exception

Question 5

Que se passe-t-il si un bloc try n'a pas de bloc except correspondant en Python ?

- Le programme plante immédiatement
- Le programme ignore l'exception
- Une exception non capturée est levée et le programme s'arrête
- Le programme exécute le bloc finally uniquement

Question 6

Tous droits réservés à STUDI - Reproduction interdite

Pourquoi utiliser des exceptions pour gérer les erreurs au lieu de simplement utiliser des conditions if-else ?

- Parce que if-else est plus lent que les exceptions
- Parce que les exceptions permettent de séparer le code normal du code de gestion des erreurs
- Parce que if-else ne peut pas gérer les erreurs
- Parce que les exceptions sont obligatoires en Python

Question 7

Quelle méthode est utilisée pour garantir que le code Python respecte les normes de style définies par la PEP 8 ?

- L'utilisation de blocs try-except
- L'installation de paquets via pip
- L'utilisation d'outils comme black ou flake8
- L'écriture de tests unitaires

Question 8

Quel est le meilleur moment pour utiliser le mot-clé raise dans un programme Python ?

- Pour démarrer un nouveau thread
- Pour forcer l'arrêt d'un programme
- Pour lever une exception personnalisée ou existante
- Pour imprimer un message de débogage

Question 9

Comment pouvez-vous intercepter plusieurs types d'exceptions dans un seul bloc except ?

- En listant les exceptions séparées par des virgules
- En utilisant plusieurs blocs except
- En listant les exceptions dans un tuple
- En utilisant un bloc try supplémentaire

Question 10

Que fait l'instruction try dans un programme Python ?

- Elle ignore les erreurs
- Elle essaie d'exécuter un bloc de code et capture les exceptions si elles se produisent
- Elle arrête le programme en cas d'erreur
- Elle débogue automatiquement le code

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

studi

Exercice p. 5 Solution n°1

Question 1

Qu'est-ce qu'une exception ?

- (●) Une condition inhabituelle qui interrompt le flux normal du programme
- ( ) Une variable non initialisée
- ( ) Une boucle infinie
- ( ) Une fonction sans paramètre

En Python, les exceptions sont utilisées pour signaler des erreurs ou des conditions anormales, comme une division par zéro, l'accès à une variable non définie ou l'accès à un fichier qui n'existe pas. Lorsqu'une exception est levée (ou 'lancée'), l'exécution du programme est interrompue et le contrôle est transféré au gestionnaire d'exceptions. Si aucune gestion d'exceptions n'est prévue, le programme se termine avec un message d'erreur.

Question 2

Quel bloc est toujours exécuté, qu'il y ait une exception ou non ?

- ( ) try
- ( ) except
- ( ) else
- (●) finally

En Python, le bloc finally est toujours exécuté, que ce soit après l'exécution réussie du bloc try ou après qu'une exception a été levée et éventuellement gérée dans un bloc except.

Question 3

Que fait le bloc `else` dans une gestion d'exception ?

- ( ) Il contient du code qui est exécuté si une exception est levée.
- (●) Il contient du code qui est exécuté si aucune exception n'est levée.
- ( ) Il contient du code qui est exécuté après le bloc `finally`.
- ( ) Il contient du code qui est exécuté avant le bloc `try`.

Le bloc else dans une gestion d'exception en Python est exécuté uniquement si aucune exception n'a été levée dans le bloc try. Autrement dit, il est utilisé pour exécuter du code qui ne doit se dérouler que lorsque le bloc try s'est terminé sans rencontrer d'erreurs.

Question 4

Quel type d'exception est levée lorsque l'on tente de convertir une chaîne non numérique en entier ?

- ( ) IOError
- (●) ValueError
- ( ) TypeError
- ( ) KeyError

Tous droits réservés à STUDI - Reproduction interdite

En Python, une ValueError est levée lorsqu'on tente d'effectuer une opération sur une donnée ayant le bon type, mais une valeur inappropriée. Par exemple, si vous essayez de convertir une chaîne de caractères non numérique, comme 'abc', en entier en utilisant int("abc"), Python lève une ValueError car 'abc' ne peut pas être interprété comme un entier..

Question 5

Pourquoi utiliser des exceptions ?

- ○ Pour arrêter le programme dès qu'une erreur survient.
- ◉ Pour gérer les erreurs de manière contrôlée et maintenir la lisibilité du code.
- ○ Pour ignorer les erreurs.
- ○ Pour augmenter la complexité du code.

Les exceptions en Python sont utilisées pour gérer les erreurs de manière contrôlée. Elles permettent au programme de réagir de manière appropriée lorsqu'une situation inattendue se produit, plutôt que de planter brusquement. En utilisant des exceptions, vous pouvez séparer le code qui traite les erreurs du code principal, ce qui améliore la lisibilité et la maintenabilité.

Exercice p. 8 Solution n°2

Question 1

Quelle est la recommandation de la PEP 8 pour l'indentation en Python ?

- ○ Utiliser 2 espaces par niveau d'indentation
- ◉ Utiliser 4 espaces par niveau d'indentation
- ○ Utiliser des tabulations pour l'indentation
- ○ Utiliser 8 espaces par niveau d'indentation

La PEP 8 recommande d'utiliser 4 espaces par niveau d'indentation pour améliorer la lisibilité du code et maintenir une structure cohérente.

Question 2

Quelle est la longueur maximale recommandée pour une ligne de code en Python selon la PEP 8 ?

- ○ 60 caractères
- ○ 72 caractères
- ◉ 79 caractères
- ○ 100 caractères

La PEP 8 recommande de limiter la longueur des lignes à 79 caractères pour éviter que les lignes ne soient coupées dans les éditeurs de texte et pour améliorer la lisibilité.

Question 3

Quel style de nommage est recommandé pour les fonctions en Python ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ camelCase
- ○ PascalCase
- ◉ snake_case
- ○ kebab-case

En Python, le style `snake_case` est recommandé pour nommer les fonctions, ce qui consiste à utiliser des minuscules avec des underscores pour séparer les mots.

Question 4

Quel est le style de nommage recommandé pour les classes en Python ?

- ○ camelCase
- ◉ PascalCase (CamelCase)
- ○ snake_case
- ○ UPPER_CASE

Le style PascalCase, également appelé CamelCase, est recommandé pour nommer les classes, en capitalisant la première lettre de chaque mot, sans underscores.

Question 5

Pourquoi est-il important d'utiliser des espaces autour des opérateurs et après les virgules ?

- ○ Pour améliorer la performance du code
- ◉ Pour respecter les conventions de syntaxe de Python
- ○ Pour améliorer la lisibilité du code
- ○ Pour réduire la longueur du code

L'utilisation d'espaces autour des opérateurs et après les virgules rend le code plus lisible et plus facile à comprendre, ce qui est une recommandation clé de la PEP 8.

Question 6

Quel est le problème avec le code suivant : `def calculersomme(a,b): return a+b` ?

- ○ Il manque des commentaires
- ○ Il utilise des tabulations au lieu d'espaces
- ◉ Il manque des espaces autour des opérateurs et des virgules
- ○ Il utilise un nom de fonction incorrect

Le code ne respecte pas la recommandation d'ajouter des espaces après les virgules et autour des opérateurs, ce qui nuit à la lisibilité.

Question 7

Dans l'exemple suivant, quel est le problème avec la définition de la classe `comptebancaire` ?

```
1 class comptebancaire:
2     def __init__(self, solde_initial):
3         self.solde = solde_initial
```

Tous droits réservés à STUDI - Reproduction interdite

- ⦿ Le nom de la classe doit être en CamelCase
- ○ Les méthodes ne sont pas correctement indentées
- ○ Le nom de la classe doit être en snake_case
- ○ Il manque un docstring pour la classe

Le nom de la classe `comptebancaire` devrait être en CamelCase, donc `CompteBancaire`, pour respecter les conventions de nommage des classes en Python.

Question 8

Quel est le principal objectif des docstrings dans le code Python ?

- ○ Décrire chaque ligne de code
- ○ Expliquer la logique du code
- ⦿ Documenter les modules, classes et fonctions
- ○ Optimiser les performances du code

Les docstrings sont utilisées pour documenter les modules, classes et fonctions en Python, fournissant une explication claire de leur objectif et de leur fonctionnement.

Question 9

Pourquoi est-il recommandé de documenter les paramètres et le type de retour dans une docstring ?

- ○ Pour améliorer la performance du code
- ⦿ Pour faciliter la compréhension et l'utilisation de la fonction par d'autres développeurs
- ○ Pour répondre aux exigences syntaxiques de Python
- ○ Pour réduire la taille de la fonction

Documenter les paramètres et le type de retour dans une docstring permet à d'autres développeurs de comprendre facilement comment utiliser la fonction et ce qu'ils doivent lui fournir en entrée.

Question 10

Dans l'exemple suivant, quel est l'objectif principal des sections "Attributes" et "Methods" dans la docstring d'une classe ?

```python
class CompteBancaire:
    """
    Une classe pour représenter un compte bancaire.

    Attributes:
        solde (float): Le solde initial du compte.

    Methods:
        deposer(montant):
            Dépose un montant sur le compte.
        retirer(montant):
            Retire un montant du compte s'il y a suffisamment de solde.
    """
```


Tous droits réservés à STUDI - Reproduction interdite

- ○ Décrire la logique interne de chaque méthode
- ◉ Fournir une documentation claire des attributs et des méthodes de la classe
- ○ Optimiser les performances de la classe
- ○ Assurer que le code est conforme à la PEP 8

Q Les sections "Attributes" et "Methods" dans la docstring d'une classe sont utilisées pour documenter les attributs de la classe et ses méthodes, ce qui aide les développeurs à comprendre ce que la classe fait et comment elle doit être utilisée.

Exercice p. 12 Solution n°3

Question 1

Quel est l'objectif principal des outils de formatage automatique en Python comme 'black' ?

- ○ Améliorer les performances du code
- ◉ Assurer la conformité du code avec la PEP 8
- ○ Identifier les bugs dans le code
- ○ Ajouter des fonctionnalités au code

Q Les outils comme 'black' sont utilisés pour formater automatiquement le code afin de le rendre conforme aux normes de style définies par la PEP 8, ce qui améliore la lisibilité et la maintenabilité du code.

Question 2

Quel est le rôle de 'isort' parmi les outils mentionnés dans la section ?

- ○ Corriger les erreurs de syntaxe
- ○ Analyser la sécurité du code
- ◉ Trier et retirer les importations inutiles
- ○ Formater le code

Q 'isort' est un outil qui permet de trier les importations dans un fichier Python et de retirer celles qui ne sont pas utilisées, ce qui aide à maintenir un code propre et organisé.

Question 3

Quel outil est recommandé pour vérifier les erreurs de style et de syntaxe en Python ?

- ○ black
- ○ bandit
- ○ isort
- ◉ flake8

Q 'flake8' est un outil qui vérifie les erreurs de style et de syntaxe dans le code Python, assurant ainsi que le code respecte les conventions de style recommandées.

Question 4

Quel outil est souvent préféré à 'flake8' pour une analyse plus complète des pratiques de codage ?

Tous droits réservés à STUDI - Reproduction interdite

- ⦿ pylint
- ○ black
- ○ isort
- ○ bandit

Q 'pylint' est souvent préféré à 'flake8' car il offre une analyse plus complète des pratiques de codage, en identifiant non seulement les erreurs de style, mais aussi les mauvaises pratiques et les problèmes potentiels dans le code.

Question 5

Comment installe-t-on 'black' pour formater le code Python ?

- ○ Via un package manager spécifique à PyCharm
- ⦿ Via pip avec la commande `pip install python-black`
- ○ Via un téléchargement manuel
- ○ Via un script Python

Q 'black' s'installe simplement via le gestionnaire de paquets pip avec la commande `pip install python-black`, ce qui permet de l'utiliser pour formater le code Python.

Question 6

Quel outil mentionné est utilisé pour effectuer des analyses de sécurité sur du code Python ?

- ○ flake8
- ○ isort
- ⦿ bandit
- ○ pylint

Q 'bandit' est un outil conçu pour analyser la sécurité du code Python, en identifiant les vulnérabilités potentielles.

Question 7

Quel est le comportement par défaut de 'black' lorsqu'il est configuré dans PyCharm ?

- ⦿ Il s'exécute automatiquement lors de l'enregistrement du fichier
- ○ Il doit être lancé manuellement via le terminal
- ○ Il formate uniquement les commentaires
- ○ Il ne s'exécute que sur demande

Q Lorsqu'il est correctement configuré dans PyCharm, 'black' s'exécute automatiquement à chaque enregistrement du fichier, ce qui garantit que le code reste formaté conformément aux standards.

Question 8

Quel est l'avantage principal d'utiliser des outils comme 'flake8' et 'pylint' ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Ils ajoutent des fonctionnalités au code
- ○ Ils réduisent la taille du fichier de code
- ◉ Ils améliorent la lisibilité et la qualité du code en identifiant les erreurs et les mauvaises pratiques
- ○ Ils accélèrent l'exécution du code

🔍 'flake8' et 'pylint' aident à améliorer la qualité du code en identifiant les erreurs de style, de syntaxe, et les mauvaises pratiques, ce qui rend le code plus lisible et maintenable.

Question 9

9. Quel est le format de la commande pour exécuter 'black' manuellement sur un fichier Python ?

- ◉ `black nom_fichier.py`
- ○ `python black nom_fichier.py`
- ○ `black --run nom_fichier.py`
- ○ `python -m black nom_fichier.py`

🔍 Pour exécuter 'black' manuellement sur un fichier Python, on utilise simplement la commande `black nom_fichier.py`.

Question 10

Comment installe-t-on les outils 'isort', 'flake8', et 'pylint' ?

- ○ Via un package manager spécifique à chaque outil
- ◉ Via pip avec la commande `pip install isort flake8 pylint`
- ○ Via un téléchargement manuel
- ○ Via une commande intégrée à Python

🔍 Les outils 'isort', 'flake8', et 'pylint' s'installent facilement via le gestionnaire de paquets pip avec la commande `pip install isort flake8 pylint`.

p. 14 **Solution n°4**

```python
# Définition de l'exception personnalisée
class DivisionParZeroException(Exception):
    def __init__(self, message="La division par zéro n'est pas autorisée."):
        super().__init__(message)
        self.message = message

# Définition de la fonction qui utilise l'exception personnalisée
def division_securisee(a, b):
    if b == 0:
        raise DivisionParZeroException()  # Levée de l'exception personnalisée si b est 0
    return a / b  # Retourne le résultat de la division si b n'est pas 0
```

Création d'une exception personnalisée (DivisionParZeroException) :

- On crée une nouvelle classe DivisionParZeroException qui hérite de la classe standard Exception. Cela permet de créer une exception spécifique à notre cas d'utilisation : interdire la division par zéro.
- Le constructeur de cette classe accepte un message (par défaut : "La division par zéro n'est pas autorisée.") qui sera affiché si cette exception est levée.

Tous droits réservés à STUDI - Reproduction interdite

Fonction division_securisee(a, b) :

Cette fonction prend deux arguments, a et b, qui représentent le numérateur et le dénominateur de la division.

Vérification du dénominateur : si b est égal à zéro, la fonction lève l'exception personnalisée DivisionParZeroException avec le message associé. Cela empêche l'exécution de la division par zéro, qui est une opération interdite en mathématiques.

Retour du résultat : si b n'est pas égal à zéro, la fonction retourne le résultat de la division a / b.

p. 14 **Solution n°5**

```
1 try:
2     # Appel de la fonction avec un dénominateur de 0 pour provoquer l'exception
3     resultat = division_securisee(10, 0)
4 except DivisionParZeroException as e:
5     # Capture de l'exception personnalisée et affichage d'un message d'erreur
6     print(f"Erreur : {e.message}")
7 else:
8     # Ce bloc ne s'exécutera pas dans ce cas puisque l'exception est levée
9     print(f"Le résultat de la division est : {resultat}")
10 finally:
11     # Ce bloc s'exécute toujours, qu'il y ait eu une exception ou non
12     print("Opération de division terminée.")
```

Explication :

- **Bloc try** :
  - Le bloc try contient le code qui pourrait potentiellement lever une exception. Ici, nous appelons la fonction division_securisee(10, 0).
  - Puisque b est égal à 0, la fonction lèvera une DivisionParZeroException.
- **Bloc except** :
  - Ce bloc capture l'exception levée par le try s'il correspond au type spécifié (ici, DivisionParZeroException).
  - Lorsque l'exception est capturée, le message d'erreur associé est affiché : "Erreur : La division par zéro n'est pas autorisée.".
- **Bloc else** :
  - Ce bloc est exécuté uniquement si aucune exception n'est levée dans le bloc try.
  - Dans ce cas, le résultat de la division serait affiché. Cependant, dans cet exemple, ce bloc ne sera pas exécuté car une exception est levée.
- **Bloc finally** :
  - Ce bloc s'exécute toujours, qu'une exception ait été levée ou non.
  - Il est utilisé ici pour afficher "Opération de division terminée.", indiquant que l'opération a été complétée, que ce soit avec succès ou avec une erreur.

Exercice p. 15 **Solution n°6**

Tous droits réservés à STUDI - Reproduction interdite

Question 1

Que signifie "ne jamais faire confiance à l'utilisateur" dans le contexte de la gestion d'erreurs ?

- ○ L'utilisateur doit être exclu du test du programme
- ◉ Les entrées de l'utilisateur doivent toujours être vérifiées et validées pour éviter les erreurs
- ○ Le programme doit bloquer toute saisie utilisateur
- ○ L'utilisateur doit être formé pour éviter les erreurs

Q Les entrées de l'utilisateur peuvent être imprévisibles. Il est donc essentiel de les vérifier et de les valider pour éviter les erreurs ou les comportements inattendus du programme.

Question 2

Comment créer une exception personnalisée en Python ?

- ○ En utilisant la classe `BaseException`
- ◉ En héritant de la classe `Exception`
- ○ En utilisant le mot-clé `custom`
- ○ En utilisant `try-except` sans `raise`

Q Pour créer une exception personnalisée en Python, on crée une nouvelle classe qui hérite de la classe `Exception` ou de l'une de ses sous-classes.

Question 3

Pourquoi utiliser un bloc `else` dans une gestion d'exceptions ?

- ○ Pour gérer les erreurs inconnues
- ◉ Pour exécuter du code seulement si aucune exception n'a été levée
- ○ Pour nettoyer les ressources après une exception
- ○ Pour remplacer le bloc `except`

Q Le bloc `else` permet d'exécuter du code seulement si le bloc `try` s'exécute sans lever d'exception.

Question 4

Dans quel cas utiliseriez-vous un bloc `finally` en Python ?

- ○ Pour ajouter une exception au programme
- ○ Pour exécuter du code uniquement si une exception est levée
- ◉ Pour exécuter du code quel que soit le résultat des blocs `try` et `except`
- ○ Pour ignorer une exception

Q Le bloc `finally` est utilisé pour exécuter du code qui doit s'exécuter dans tous les cas, que ce soit après une exception ou non, comme la fermeture de fichiers ou la libération de ressources.

Question 5

Que se passe-t-il si un bloc try n'a pas de bloc except correspondant en Python ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Le programme plante immédiatement
- ○ Le programme ignore l'exception
- ◉ Une exception non capturée est levée et le programme s'arrête
- ○ Le programme exécute le bloc finally uniquement

Q Si une exception est levée dans un bloc try sans bloc except correspondant pour la capturer, l'exception est non capturée et le programme s'arrête avec un message d'erreur.

Question 6

Pourquoi utiliser des exceptions pour gérer les erreurs au lieu de simplement utiliser des conditions if-else ?

- ○ Parce que if-else est plus lent que les exceptions
- ◉ Parce que les exceptions permettent de séparer le code normal du code de gestion des erreurs
- ○ Parce que if-else ne peut pas gérer les erreurs
- ○ Parce que les exceptions sont obligatoires en Python

Q Les exceptions permettent de séparer clairement le code fonctionnel du code de gestion des erreurs, rendant le code plus lisible et plus facile à maintenir.

Question 7

Quelle méthode est utilisée pour garantir que le code Python respecte les normes de style définies par la PEP 8 ?

- ○ L'utilisation de blocs try-except
- ○ L'installation de paquets via pip
- ◉ L'utilisation d'outils comme black ou flake8
- ○ L'écriture de tests unitaires

Q Des outils comme black pour le formatage automatique et flake8 pour la vérification des erreurs de style sont utilisés pour garantir que le code respecte les normes de style définies par la PEP 8.

Question 8

Quel est le meilleur moment pour utiliser le mot-clé raise dans un programme Python ?

- ○ Pour démarrer un nouveau thread
- ○ Pour forcer l'arrêt d'un programme
- ◉ Pour lever une exception personnalisée ou existante
- ○ Pour imprimer un message de débogage

Q Le mot-clé raise est utilisé pour lever une exception, qu'il s'agisse d'une exception personnalisée ou d'une exception Python standard, permettant ainsi de signaler qu'une condition d'erreur a été rencontrée.

Question 9

Comment pouvez-vous intercepter plusieurs types d'exceptions dans un seul bloc except ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ En listant les exceptions séparées par des virgules
- ○ En utilisant plusieurs blocs except
- ◉ En listant les exceptions dans un tuple
- ○ En utilisant un bloc try supplémentaire

Pour intercepter plusieurs types d'exceptions dans un seul bloc except, vous pouvez les lister dans un tuple. Par exemple : except (ValueError, ZeroDivisionError):.

Question 10

Que fait l'instruction try dans un programme Python ?

- ○ Elle ignore les erreurs
- ◉ Elle essaie d'exécuter un bloc de code et capture les exceptions si elles se produisent
- ○ Elle arrête le programme en cas d'erreur
- ○ Elle débogue automatiquement le code

L'instruction try permet d'exécuter un bloc de code tout en capturant les exceptions éventuelles, permettant ainsi de gérer les erreurs sans interrompre brusquement le programme.

Tous droits réservés à STUDI - Reproduction interdite

Tous droits réservés à STUDI - Reproduction interdite